

SPEC Técnica — Sistema de Gestão de Esportes e Lazer (Polo CSU)

Stack: HTML5 + CSS3 + JavaScript (ES Modules) + Supabase

Arquitetura: SPA leve sem build step. Todas as dependências via CDN ou arquivo local.

Público-alvo: Agente de IA / Desenvolvedor responsável pela implementação.

1. Visão Geral do Sistema

Aplicação web responsiva para centralização da gestão operacional da Secretaria de Esporte e Lazer. O sistema deve permitir o cadastro de alunos, professores e modalidades; controle de turmas e matrículas; registro de frequência (com alertas automáticos); e geração de relatórios operacionais.

Restrições técnicas:

- Sem framework de build (Vite, Webpack, etc.).
- Sem framework SPA pesado (React, Vue, Angular).
- Toda funcionalidade deve funcionar offline-first quando possível, sincronizando com Supabase quando online.

2. Stack e Dependências

Biblioteca	Versão	Propósito	Fonte
Supabase JS	^2.x	Cliente oficial. Auth, Banco (PostgREST), Realtime, Storage	CDN / npm local
Alpine.js	^3.x	Reatividade declarativa, state management, comportamentos de UI sem build step	CDN
Tailwind CSS	^3.x	Utility-first CSS, design system consistente, manutenção mínima	CDN (Play CDN)
Chart.js	^4.x	Gráficos operacionais (ocupação, evasão, frequência)	CDN
Tom Select	^2.x	Selects avançados com busca (modalidades, alunos, professores)	CDN
Flatpickr	^4.x	Datepicker acessível para datas de nascimento, aulas	CDN
QRCode.js	^1.x	Geração de QR Code para check-in do aluno	CDN

Table 1 – continued

Biblioteca	Versão	Propósito	Fonte
html5-qrcode	^2.x	Leitura de QR Code via câmera (check-in do professor)	CDN
SheetJS (xlsx)	^0.18.x	Exportação de relatórios para Excel	CDN
Phosphor Icons	^2.x	Ícones consistentes (peso regular padrão)	CDN

2.1 Estrutura de Arquivos

```

/polo-csu/
├─ index.html          # Login / Dashboard
├─ app/
│  ├─ js/
│  │  ├─ main.js       # Entry point: inicializa Alpine, Supabase, rotas
│  │  ├─ config.js     # Constantes e configuração do Supabase
│  │  ├─ db.js         # Wrapper do Supabase (queries, auth, realtime)
│  │  ├─ store.js      # Estado global Alpine (usuário logado, notificações)
│  │  ├─ router.js     # Roteamento simples (hash-based)
│  │  ├─ utils.js      # Helpers (máscaras, validações, formatters)
│  │  └─ components/   # Componentes reutilizáveis Alpine
│  ├─ css/
│  │  └─ custom.css    # Overrides mínimos do Tailwind
│  └─ pages/
│     ├─ dashboard.html
│     ├─ alunos.html
│     ├─ professores.html
│     ├─ modalidades.html
│     ├─ turmas.html
│     ├─ matriculas.html
│     ├─ frequencia.html
│     └─ relatorios.html
└─ assets/
   └─ logo-prefeitura.svg

```

3. Arquitetura

3.1 Padrão: MPA com comportamento SPA

- Navegação entre páginas via **carregamento assíncrono** (fetch do HTML da página + injeção no container).
- Roteamento baseado em hash: `#/dashboard`, `#/turmas`, `#/frequencia`.
- Cada página é um arquivo HTML independente com seu próprio `<script>` e `x-data` do Alpine.

3.2 Comunicação com Supabase

- Todas as operações de banco passam pelo arquivo `db.js`.
- Uso de **Row Level Security (RLS)** no Supabase para isolamento de dados.
- **Realtime** habilitado nas tabelas `presenca` e `matricula` para atualização em tempo real das telas de frequência e lista de espera.

3.3 Estado Global (Alpine Store)

```
// store.js — registrado no Alpine
Alpine.store('app', {
  user: null,           // Sessão atual (perfil, role)
  notificacoes: [],     // Toast notifications
  offline: false,       // Status de conectividade
  turmaAtiva: null,     // Contexto de frequência
})
```

4. Modelo de Dados (Supabase / PostgreSQL)

4.1 Tabelas Principais

```
-- perfis (estende auth.users do Supabase)
create table public.perfis (
  id uuid references auth.users on delete cascade primary key,
  nome text not null,
  email text not null,
  telefone text,
  role text not null check (role in ('admin', 'gestor', 'professor', 'recepcao')),
  polo text default 'CSU',
  created_at timestamptz default now()
);

-- modalidades
create table public.modalidades (
  id uuid default gen_random_uuid() primary key,
  nome text not null unique,
  descricao text,
  ativa boolean default true,
  created_at timestamptz default now()
);

-- professores
create table public.professores (
  id uuid default gen_random_uuid() primary key,
  perfil_id uuid references public.perfis(id),
  modalidade_id uuid references public.modalidades(id),
  cref text, -- registro profissional, se houver
  created_at timestamptz default now()
);
```

```

-- alunos
create table public.alunos (
    id uuid default gen_random_uuid() primary key,
    nome text not null,
    cpf text unique,
    data_nascimento date,
    telefone text,
    email text,
    responsavel_nome text, -- obrigatório se menor de idade
    responsavel_telefone text,
    endereco text,
    atestado_medico_url text, -- URL no Storage
    atestado_valido_ate date,
    observacoes text,
    created_at timestamptz default now()
);

-- turmas
create table public.turmas (
    id uuid default gen_random_uuid() primary key,
    modalidade_id uuid not null references public.modalidades(id),
    professor_id uuid references public.professores(id),
    nome text not null, -- ex: "Natação Adulto - Ter/Qui 19h"
    horario text not null, -- ex: "19:00 - 20:00"
    dias_semana text[] not null, -- ['seg','qua','sex']
    vagas_total int not null default 0,
    vagas_disponiveis int not null default 0,
    idade_min int,
    idade_max int,
    ativa boolean default true,
    created_at timestamptz default now()
);

-- matriculas
create table public.matriculas (
    id uuid default gen_random_uuid() primary key,
    aluno_id uuid not null references public.alunos(id),
    turma_id uuid not null references public.turmas(id),
    status text not null check (status in ('ativa','cancelada','lista_espera','concluida')),
    data_matricula date default current_date,
    data_cancelamento date,
    motivo_cancelamento text,
    created_at timestamptz default now(),
    unique(aluno_id, turma_id)
);

-- presenca
create table public.presenca (
    id uuid default gen_random_uuid() primary key,
    matricula_id uuid not null references public.matriculas(id),
    data_aula date not null,

```

```

    presente boolean not null default false,
    checkin_por uuid references public.perfis(id), -- quem registrou
    checkin_metodo text check (checkin_metodo in ('manual','qr','lista')),
    observacao text,
    created_at timestamptz default now(),
    unique(matricula_id, data_aula)
);

-- configuracoes (alertas de falta)
create table public.configuracoes (
    id uuid default gen_random_uuid() primary key,
    chave text not null unique,
    valor text not null,
    descricao text
);
-- Seed: ('alerta_faltas_consecutivas', '3', 'Número de faltas para disparar alerta')

```

4.2 Storage Buckets

- atestados: PDFs dos atestados médicos (pasta privada, acesso via RLS).

4.3 Row Level Security (RLS) — Regras Obrigatórias

Tabela	Política
perfis	Usuário lê próprio perfil. Admin/Gestor lê todos.
alunos	Todos os perfis autenticados podem ler/inserir. Apenas Admin/Gestor editam.
turmas	Todos leem. Apenas Admin/Gestor editam.
matriculas	Professor lê matrículas de suas turmas. Admin/Gestor leem tudo.
presenca	Professor insere/emenda presença de suas turmas. Admin/Gestor leem tudo.

5. Funcionalidades por Módulo

5.1 Autenticação (#/login)

- Login via e-mail/senha (Supabase Auth).
- Recuperação de senha por e-mail.
- Redirecionamento automático para #/dashboard após login.
- Logout limpa localStorage e estado global.

5.2 Dashboard (#/dashboard)

- Cards KPI: total de alunos ativos, turmas com vagas, taxa de frequência do dia, alertas de faltas pendentes.
- Gráfico Chart.js: frequência dos últimos 7 dias (linha) e ocupação por modalidade (barra).
- Lista de alertas recentes (alunos com 3+ faltas consecutivas).

5.3 Cadastro de Alunos (#/alunos)

- Formulário com validação em tempo real (CPF, e-mail, idade).
- Upload de atestado médico para Storage.
- Busca instantânea por nome ou CPF.
- Indicador visual: atestado vencido (vermelho), válido (verde), não enviado (cinza).
- Modal de detalhes com histórico de matrículas.

5.4 Cadastro de Professores (#/professores)

- Vinculação a perfis (usuário do sistema) e modalidades.
- Lista de turmas sob responsabilidade.

5.5 Modalidades (#/modalidades)

- CRUD simples. Campo ativa para desativar sem excluir histórico.

5.6 Turmas (#/turmas)

- CRUD com selects dependentes (modalidade → professor).
- Cálculo automático de `vagas_disponiveis`: `vagas_total - count(matriculas_ativas)`.
- Visualização semanal (grade horária) por polo.
- Indicador de turma lotada (vermelho), disponível (verde).

5.7 Matrículas (#/matriculas)

- Fluxo de inscrição: seleciona aluno → seleciona turma → valida idade/atestado/vagas.
- Se `vagas_disponiveis > 0`: status ativa.
- Se `vagas_disponiveis == 0`: status lista_espera + notificação automática.
- Transferência entre turmas (cancela origem, cria nova matrícula).
- Filtros: por modalidade, status, período.

5.8 Frequência (#/frequencia)

- **Modo Professor:** lista de alunos da turma do dia. Check/uncheck rápido. Salvamento em lote.
- **Modo QR Code:** gera QR único por aluno (exibe em tela para foto); professor escaneia com câmera (html5-qrcode).
- **Modo Lista:** importação rápida de lista pré-preenchida.
- Registro automático de `checkin_por` e `checkin_metodo`.
- Após salvamento, trigger calcula faltas consecutivas do aluno.

5.9 Relatórios (#/relatorios)

- Filtros: modalidade, turma, período.
- Relatórios disponíveis:
 1. **Frequência por Turma** (tabela + gráfico de heatmap).
 2. **Alunos com Alerta de Faltas** (lista para ação de gestor).
 3. **Ocupação de Vagas** (por modalidade, com lista de espera).
 4. **Evasão** (matrículas canceladas por período, com motivo).
- Exportação para Excel (.xlsx) via SheetJS.
- Exportação para PDF via `window.print()` com CSS @media print otimizado.

6. Regras de Negócio Automatizadas (Triggers/Functions)

Implementar como **PostgreSQL Functions** no Supabase:

Regra	Implementação
Atualizar vagas disponíveis	Trigger AFTER INSERT/UPDATE/DELETE em matriculas recalcula turmas.vagas_disponiveis.
Alerta de faltas	Trigger AFTER INSERT em presenca: se aluno tem 3 registros presente = false consecutivos na mesma turma, insere registro na tabela alertas_faltas (ou envia notificação).
Validação de idade	Function chamada no INSERT de matriculas: rejeita se idade do aluno não estiver entre turmas.idade_min e turmas.idade_max.
Atestado obrigatório	Function valida se modalidade exige atestado (ex: natação, hidroginástica) e se aluno.atestado_valido_ate >= current_date.
Desativação em lote	Agendamento (pg_cron ou edge function) para cancelar matrículas lista_espera com mais de 30 dias sem vaga.

7. Interface e UX

7.1 Design System (Tailwind)

- **Cores:**
 - Primária: blue-700 (confiança, institucional).
 - Sucesso: emerald-500.
 - Alerta: amber-500.
 - Erro: rose-500.
 - Fundo: slate-50.
- **Tipografia:** Fonte do sistema (font-sans), tamanho base 16px.
- **Componentes base:**
 - Botões: rounded-lg, px-4 py-2, transição 150ms.
 - Tabelas: striped, hover slate-100, cabeçalho fixo.
 - Cards: shadow-sm, border border-slate-200, rounded-xl.
 - Modais: overlay bg-black/50, centralizado, max-w-lg.

7.2 Responsividade

- **Desktop:** sidebar fixa à esquerda (256px), conteúdo fluido.
- **Mobile (< 768px):** bottom navigation ou menu hamburger, tabelas com scroll horizontal, cards empilhados.

7.3 Acessibilidade

- Todos os inputs com <label> associado.
- Contraste mínimo 4.5:1.
- Foco visível (ring-2 ring-blue-500).

- Mensagens de erro próximas ao campo.

8. Autenticação e Autorização

8.1 Perfis de Acesso

Perfil	Permissões
Admin	Acesso total. Gerencia usuários, configurações, todos os cadastros.
Gestor	CRUD de turmas, matrículas, relatórios. Não gerencia usuários do sistema.
Professor	Visualiza suas turmas, registra frequência, vê alertas de faltas de seus alunos.
Recepção	Cadastra alunos, realiza matrículas, não acessa relatórios estratégicos.

8.2 Controle de Acesso no Frontend

- Roteador (`router.js`) verifica `store.user.role` antes de renderizar página.
- Páginas restritas redirecionam para `#/dashboard` com toast de erro.

9. LGPD e Segurança

Requisito	Implementação
Consentimento	Checkbox obrigatório no cadastro de aluno: “Autorizo o uso dos dados para fins de gestão esportiva”.
Dados sensíveis	Atestado médico armazenado em Storage com RLS restrito. Campo <code>observacoes</code> criptografado ou com acesso limitado.
Menores de idade	Campos <code>responsavel_nome</code> e <code>responsavel_telefone</code> obrigatórios se <code>data_nascimento</code> calcula < 18 anos.
Exclusão lógica	Nunca usar DELETE em alunos; usar flag <code>ativo = false</code> ou arquivar.
Logs de auditoria	Tabela <code>auditoria</code> (opcional no MVP): registra quem alterou o quê e quando.
Senhas	Gerenciadas pelo Supabase Auth (hash+bcrypt, política de força mínima).

10. Estratégia Offline-First

- Usar **Service Worker** básico para cache das páginas e assets estáticos.
- Operações de escrita (presença, matrícula) armazenadas em `localStorage` fila de sincronização.
- Quando online, `db.js` consome a fila e sincroniza com Supabase, mostrando badge de “Sincronizando...” / “Sincronizado”.

- Supabase client configurado com `db.realtime` para reconexão automática.

11. Critérios de Aceite (Definition of Done)

- Login funcional com redirecionamento por perfil.
- CRUD completo de alunos, professores, modalidades e turmas.
- Matrícula com controle automático de vagas e lista de espera.
- Registro de frequência em 3 cliques por aluno.
- Alerta de faltas visível no dashboard em até 5 segundos após registro.
- Relatórios exportáveis em Excel e impressos em PDF.
- Aplicação responsiva em tablet (uso em campo) e desktop (uso na secretaria).
- RLS ativo em todas as tabelas; usuário não autenticado não acessa dados.
- Funcionamento básico offline (cache + fila de sincronização).
- Documentação de deploy (variáveis de ambiente, seeds SQL) no `README.md`.

12. Variáveis de Ambiente

Arquivo `.env` (não versionado; exposto no build apenas as não-sensíveis):

```
SUPABASE_URL=https://xxxx.supabase.co
SUPABASE_ANON_KEY=eyJ...
```

No frontend puro, variáveis podem ser injetadas em `config.js` no momento do deploy ou via substituição de string no CI.

13. Notas para o Agente de IA

1. **Priorize consistência:** use os mesmos padrões de nomenclatura (camelCase JS, snake_case SQL, kebab-case HTML IDs).
2. **Evite lógica no HTML:** mantenha o `x-data` enxuto; delegue para funções em `db.js` ou `utils.js`.
3. **Tratamento de erro:** toda chamada Supabase deve ter `.catch()` e exibir toast via `Alpine.store('app').notificacoes`.
4. **Performance:** use `select()` enxuto no Supabase; evite `*` em tabelas grandes.
5. **Testabilidade:** isole funções puras em `utils.js` (ex: `calcularIdade`, `validarCPF`, `formatarTelefone`).
6. **Não reinvente:** se uma biblioteca da lista resolve o problema, use-a. Não escreva parser de Excel próprio, por exemplo.

Fim da SPEC.